

ETC5513 Assignment 2 Guide

ETC5513 Assignment 2 Guide

1. Creating an RStudio Project and Rendering a Quarto File

What I did

I first created a new RStudio Project called `etc5513-assignment2`. This project folder was used as the working directory for the assignment. Using an RStudio Project helps keep all files for the task in one place, which supports reproducibility and makes it easier to manage files with Git.

Inside this project folder, I created a Quarto file called `example.qmd`. The file contained a title, a short markdown section, and a simple R code chunk using the built-in `cars` dataset with the command `summary(cars)`.

Why I did this

The purpose of this step was to create a simple reproducible document that could be rendered into an HTML file. Quarto combines markdown text and executable R code, so the output can be regenerated directly from the source file.

I used the built-in `cars` dataset because it is included in base R and does not require additional packages or external data files.

Result

I rendered `example.qmd` into an HTML file using the Render button in RStudio. After rendering, the project folder contained both:

- `example.qmd`
- `example.html`

This showed that the Quarto document was successfully knitted into an HTML file.

Evidence

The screenshot below shows the rendered HTML output generated from `example.qmd`.

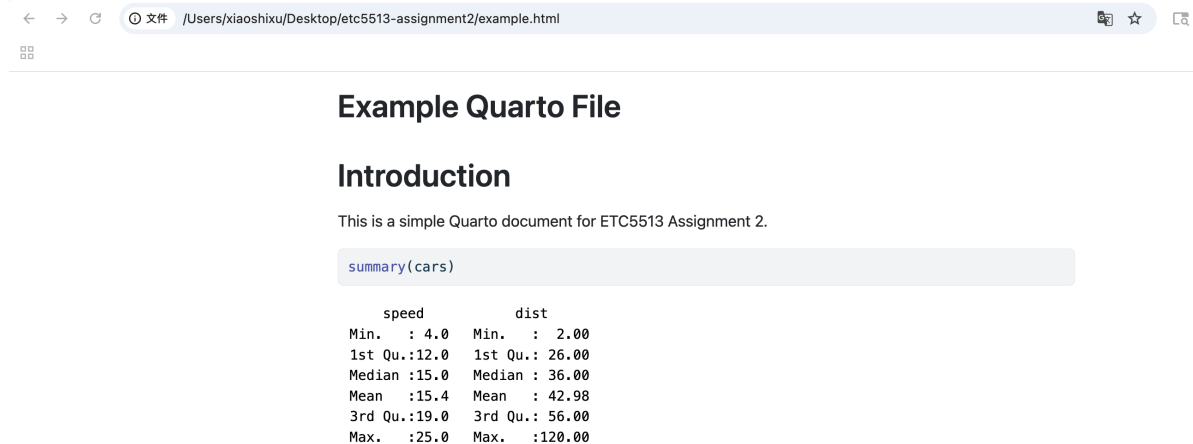


Figure 1: Rendered HTML output

2. Initialising a Git Repository and Pushing to GitHub

What I did

After creating the project files, I used the command line interface in RStudio to initialise the project folder as a Git repository. I first checked the current working directory using `pwd` and listed the project files using `ls` to confirm that I was inside the correct project folder.

I then initialised Git using the `git init` command. After initialisation, I used `git status` to inspect the repository state and identify untracked files.

Before committing the files, I created a `.gitignore` file to exclude machine-specific and temporary files such as `.DS_Store`, `.Rhistory`, and `.Rproj.user/`. This helps keep the repository clean and improves reproducibility across different computers.

Next, I added the project files to the staging area using `git add .` and checked the repository status again with `git status`.

I then created the first commit using the following clear commit message:

`Add initial Quarto project files`

After the local repository was prepared, I created a new repository on GitHub and connected it to the local repository using:

```
git remote add origin
```

Finally, I pushed the local `main` branch to GitHub using:

```
git push -u origin main
```

Why I did this

The purpose of these steps was to establish version control for the project and connect the local repository to a remote GitHub repository.

Using Git allows project changes to be tracked through commits, making it possible to manage project history and collaborate more effectively. The `.gitignore` file prevents unnecessary local system files from being tracked.

The commit process followed the standard Git workflow:

- working directory
- staging area
- local repository
- remote repository

Using a clear commit message improves the readability of the repository history and makes the workflow easier to understand.

Result

The repository was successfully initialised and committed locally. After pushing to GitHub, the remote repository contained the project files, including:

- `assignment2_guide.qmd`
- `example.qmd`
- `example.html`
- `screenshots/`

The terminal output also confirmed that the local `main` branch was successfully linked to `origin/main`.

3. Creating and Using a New Git Branch

What I did

I created a new Git branch called `testbranch` using the command:

```
git switch -c testbranch
```

I then checked the available branches using `git branch` to confirm that the current branch had successfully switched from `main` to `testbranch`.

After switching branches, I modified the file `example.qmd` by changing the introductory text. The updated text explained that the file had been modified on `testbranch` to demonstrate independent development using Git branches.

I then checked the repository status using `git status` to confirm that `example.qmd` had been modified.

Next, I added the modified file to the staging area using:

```
git add example.qmd
```

After checking the repository status again, I created a new commit using the following commit message:

```
Modify example file on testbranch
```

Finally, I pushed the new branch to the remote GitHub repository using:

```
git push -u origin testbranch
```

Why I did this

The purpose of this step was to demonstrate how Git branches can be used to develop changes independently from the `main` branch.

Branches allow multiple versions of a project to exist simultaneously without affecting the main project history. This makes experimentation and collaborative development safer and easier.

The workflow followed the standard Git branching process:

- create a branch
- switch to the branch
- modify files
- stage changes
- commit changes
- push the branch to GitHub

Using a separate branch also prepares the repository for future merging and conflict resolution workflows.

Result

The new branch `testbranch` was successfully created both locally and on GitHub. The modifications to `example.qmd` were committed and pushed successfully.

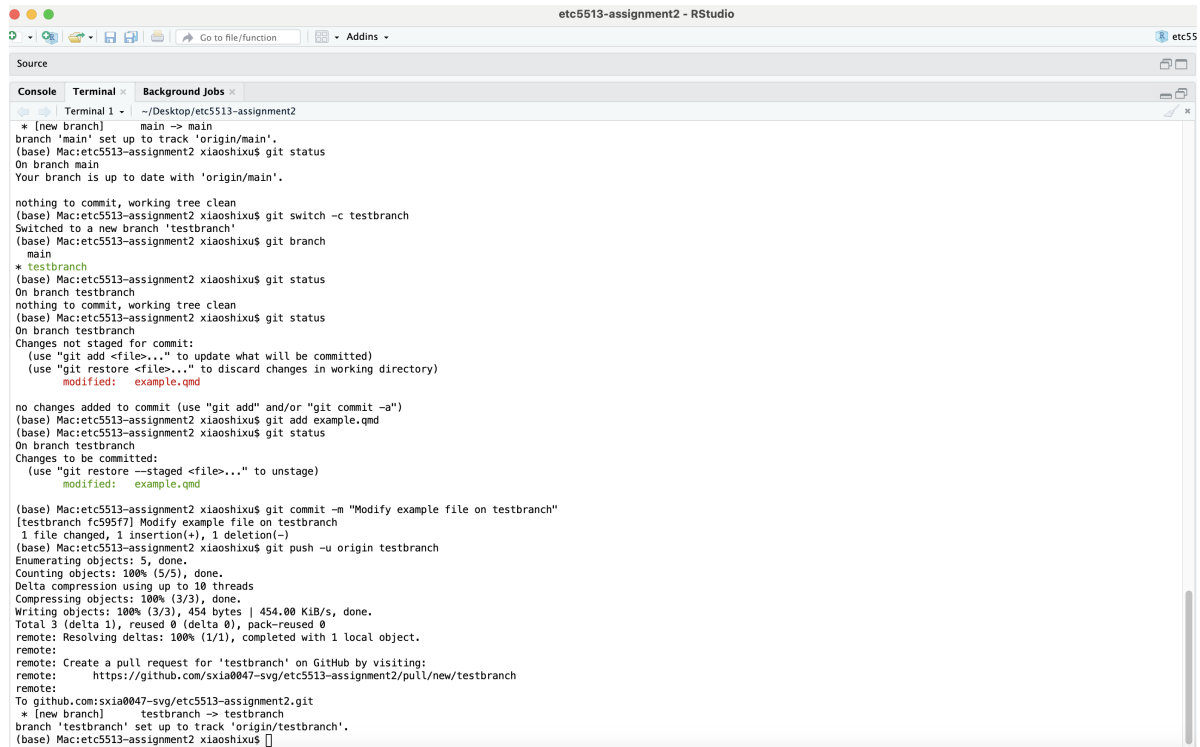
The terminal output confirmed that the local branch was linked to the remote branch `origin/testbranch`.

Evidence

Creating, modifying, and pushing testbranch

The screenshot below shows the complete Git branching workflow in the terminal, including:

- creating and switching to `testbranch`
- modifying `example.qmd`
- staging the modified file
- committing the changes with a clear commit message
- pushing the branch to GitHub



```
etc5513-assignment2 - RStudio
Source
Console Terminal Background Jobs
Terminal 1 ~ /Desktop/etc5513-assignment2
* [new branch] main -> main
branch 'main' set up to track 'origin/main'.
(base) Mac:etc5513-assignment2 xiaoshixu$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
(base) Mac:etc5513-assignment2 xiaoshixu$ git switch -c testbranch
Switched to a new branch 'testbranch'
(base) Mac:etc5513-assignment2 xiaoshixu$ git branch
  main
* testbranch
(base) Mac:etc5513-assignment2 xiaoshixu$ git status
On branch testbranch
nothing to commit, working tree clean
(base) Mac:etc5513-assignment2 xiaoshixu$ git status
On branch testbranch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   example.qmd

no changes added to commit (use "git add" and/or "git commit -a")
(base) Mac:etc5513-assignment2 xiaoshixu$ git add example.qmd
(base) Mac:etc5513-assignment2 xiaoshixu$ git status
On branch testbranch
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   example.qmd

(base) Mac:etc5513-assignment2 xiaoshixu$ git commit -m "Modify example file on testbranch"
[testbranch fc595f7] Modify example file on testbranch
1 file changed, 1 insertion(+), 1 deletion(-)
(base) Mac:etc5513-assignment2 xiaoshixu$ git push -u origin testbranch
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 454 bytes | 454.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'testbranch' on GitHub by visiting:
remote:   https://github.com/sxia0047-svg/etc5513-assignment2/pull/new/testbranch
remote:
To github.com:sxia0047-svg/etc5513-assignment2.git
* [new branch] testbranch -> testbranch
branch 'testbranch' set up to track 'origin/testbranch'.
(base) Mac:etc5513-assignment2 xiaoshixu$
```

Figure 2: Git branch workflow

4. Adding a Data Folder and Amending a Commit

What I did

While working on `testbranch`, I created a new folder called `data` using the command:

```
mkdir data
```

I then added the Assignment 1 dataset into the `data` folder.

After adding the data file, I checked the repository status using `git status` to confirm that Git had detected the new folder and file as untracked content.

Next, I added the `data` folder to the staging area using:

```
git add data/
```

After confirming that the new data file had been staged successfully, I amended the previous commit using:

```
git commit --amend -m "Add guide section for testbranch workflow and assignment data"
```

This updated the previous commit so that the Assignment 1 data file became part of the existing commit history instead of creating a separate additional commit.

Because amending a commit rewrites Git history, I then updated the remote branch using:

```
git push --force-with-lease origin testbranch
```

After completing the amend workflow, I added a screenshot documenting the process and committed the updated guide files using the following commit message:

```
Add guide section for amend workflow
```

Finally, I pushed the updated branch to GitHub to synchronise the latest project documentation.

Why I did this

The purpose of this step was to demonstrate how Git can update existing commit history using the amend workflow.

The **data** folder was created to store project data files in a structured and reproducible way. Organising files into dedicated folders improves project management and makes repositories easier to understand.

Using `git commit --amend` allowed the previous commit to be modified instead of creating a new commit. This is useful when small changes or missing files need to be added to the most recent commit.

Because amending a commit changes Git history, a force push was required to update the remote repository. I used:

```
git push --force-with-lease
```

instead of a normal force push because it is a safer workflow that checks for unexpected remote changes before overwriting the remote branch history.

Updating and committing the assignment guide also demonstrated how documentation and screenshots can be version controlled alongside project files.

Result

The **data** folder and Assignment 1 dataset were successfully added to the repository. The previous commit was amended successfully, and the updated commit history was pushed to GitHub.

The terminal output confirmed that the remote branch **testbranch** was updated using a forced update after the amended commit.

The updated assignment guide and screenshots were also committed and synchronised with the GitHub repository.

5. Modifying the Main Branch to Create a Merge Conflict

What I did

After completing the work on **testbranch**, I switched back to the **main** branch using:

```
git switch main
```

I then checked the available branches using `git branch` and confirmed that the current branch had successfully changed back to **main**.

Next, I checked the repository status using `git status` to confirm that the working tree was clean before making further changes.

After returning to **main**, I modified the file **example.qmd** by changing the same section that had previously been modified on **testbranch**. However, the text was changed in a different way so that the modifications on **main** would conflict with the version on **testbranch** during a future merge.

I then checked the repository status again using `git status` to confirm that the file had been modified.

Next, I staged the modified file using:

```
git add example.qmd
```

After confirming that the file had been staged successfully, I created a new commit using the following commit message:

```
Modify example file on main
```

Finally, I pushed the updated **main** branch to the remote GitHub repository using:

```
git push origin main
```


Why I did this

The purpose of this step was to demonstrate how merge conflicts can occur when two branches modify the same section of the same file in different ways.

Git branches allow independent development, but conflicts can occur when changes from separate branches cannot be merged automatically. By intentionally modifying the same part of `example.qmd` differently on `main` and `testbranch`, a future merge conflict was intentionally created for demonstration purposes.

This workflow also demonstrated the standard Git process for updating a branch:

- switch branches
- modify files
- stage changes
- commit changes
- push changes to GitHub

Using a clear commit message also improves repository history and makes project changes easier to understand.

Result

The `main` branch was successfully updated locally and on GitHub. The modified version of `example.qmd` was committed and pushed successfully.

The repository now contains different versions of the same file on `main` and `testbranch`, which will create a merge conflict during the next merge operation.

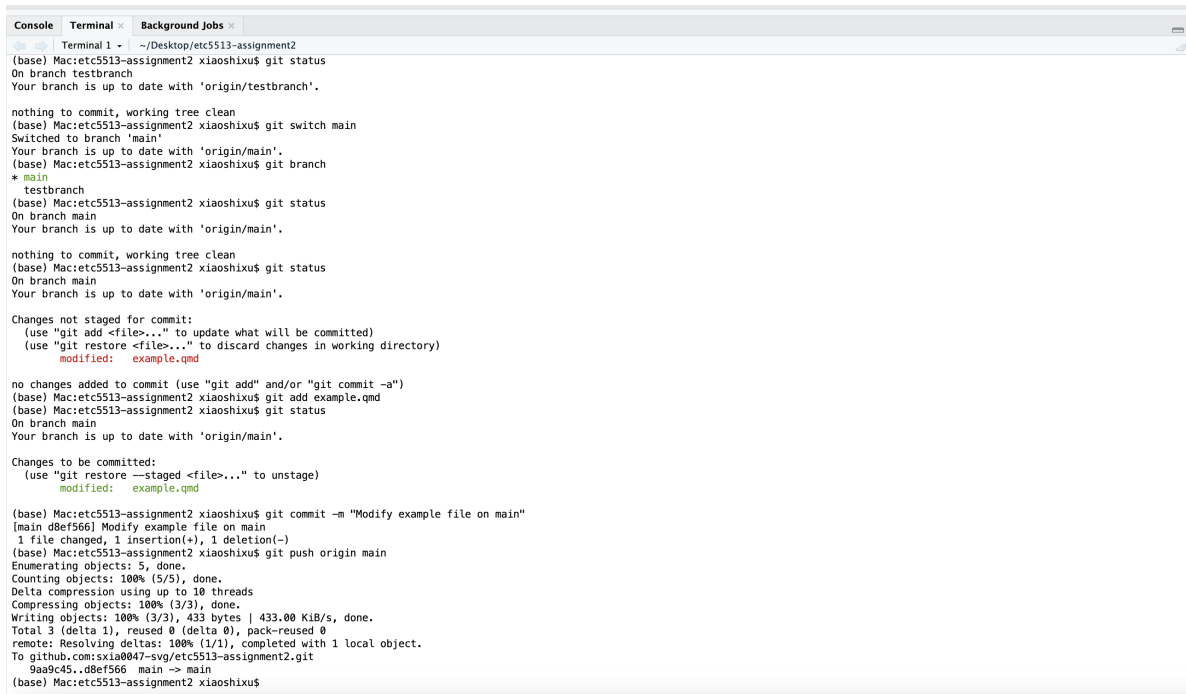
Evidence

Modifying the `main` branch

The screenshot below shows the workflow for:

- switching from `testbranch` back to `main`
- checking the repository status
- modifying `example.qmd`
- staging the modified file
- committing the changes with a clear commit message
- pushing the updated `main` branch to GitHub

The terminal output also confirms that the updated `main` branch was successfully synchronised with the remote repository.



```
Console Terminal Background Jobs
Terminal 1 ~ ~/Desktop/etc5513-assignment2
(base) Mac:etc5513-assignment2 xiaoshixu$ git status
On branch testbranch
Your branch is up to date with 'origin/testbranch'.

nothing to commit, working tree clean
(base) Mac:etc5513-assignment2 xiaoshixu$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
(base) Mac:etc5513-assignment2 xiaoshixu$ git branch
* main
  testbranch
(base) Mac:etc5513-assignment2 xiaoshixu$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
(base) Mac:etc5513-assignment2 xiaoshixu$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   example.qmd

no changes added to commit (use "git add" and/or "git commit -a")
(base) Mac:etc5513-assignment2 xiaoshixu$ git add example.qmd
(base) Mac:etc5513-assignment2 xiaoshixu$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   example.qmd

(base) Mac:etc5513-assignment2 xiaoshixu$ git commit -m "Modify example file on main"
[main d8ef566] Modify example file on main
1 file changed, 1 insertion(+), 1 deletion(-)
(base) Mac:etc5513-assignment2 xiaoshixu$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 433 bytes | 433.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:sxia0047-svq/etc5513-assignment2.git
   9aa9c45..d8ef566  main -> main
(base) Mac:etc5513-assignment2 xiaoshixu$
```

Figure 3: Modifying main branch

6. Merging testbranch into main and Resolving a Merge Conflict

What I did

While on the `main` branch, I merged the changes from `testbranch` using:

```
git merge testbranch
```

Git detected merge conflicts because both `main` and `testbranch` had modified the same part of `example.qmd` in different ways. The terminal also showed conflicts in `assignment2_guide.qmd` and `assignment2_guide.pdf`.

I checked the conflict status using:

```
git status
```

The output showed unmerged paths, including:

- `assignment2_guide.pdf`
- `assignment2_guide.qmd`

- `example.qmd`

For the guide files, I kept the version from `testbranch` because it contained the more complete guide sections from earlier steps. I used:

```
git checkout --theirs assignment2_guide.qmd
```

```
git checkout --theirs assignment2_guide.pdf
```

Then I manually opened `example.qmd` and fixed the merge conflict by removing the conflict markers and replacing the conflicting text with a resolved final version.

After fixing the conflict, I staged the resolved files using:

```
git add assignment2_guide.qmd assignment2_guide.pdf example.qmd
```

I then checked the status again. Git confirmed that all conflicts had been fixed and the repository was ready to complete the merge.

Finally, I completed the merge commit and pushed the updated `main` branch to GitHub using:

```
git commit -m "Resolve merge conflict from testbranch"
```

```
git push origin main
```

Why I did this

The purpose of this step was to demonstrate how Git handles merge conflicts and how conflicts can be resolved manually.

A merge conflict occurs when Git cannot automatically combine changes from two branches. In this case, both `main` and `testbranch` modified the same section of `example.qmd`, so Git required manual conflict resolution.

Using `git status` during the merge helped identify which files had conflicts and which files were ready to be committed. This follows good Git practice because `git status` gives a clear overview of the repository state during conflict resolution.

For the guide files, I used the `testbranch` version because it contained the completed guide sections. For `example.qmd`, I manually edited the file to remove the conflict markers and create a clean final version.

This workflow demonstrates the standard merge conflict process:

- merge one branch into another
- identify conflicts
- inspect conflicted files
- resolve conflicts manually or by choosing a branch version

- stage resolved files
- create a merge commit
- push the resolved merge to GitHub

Result

The changes from `testbranch` were successfully merged into `main`. The merge conflict was resolved, and the final version of `example.qmd` no longer contained conflict markers.

The updated `main` branch was pushed to GitHub successfully. The repository now contains the combined work from both branches, including the guide sections, screenshots, data folder, and resolved `example.qmd` file.

7. Creating and Pushing an Annotated Git Tag

What I did

After completing the merge workflow on the `main` branch, I first checked the repository status using:

```
git status
```

The output confirmed that the working tree was clean and that the local `main` branch was fully synchronised with the remote repository.

I then created an annotated Git tag called `v1.0` using:

```
git tag -a v1.0 -m "Version 1.0 release"
```

After creating the tag, I checked the available tags using:

```
git tag
```

The output confirmed that the `v1.0` tag had been created successfully.

Next, I inspected the annotated tag information using:

```
git show v1.0
```

This displayed the tag metadata, including:

- the tag name
- the tagger information
- the tag message
- the commit associated with the tag

Finally, I pushed the annotated tag to GitHub using:

```
git push origin v1.0
```

Why I did this

The purpose of this step was to demonstrate how Git tags can be used to mark important project versions and releases.

Annotated tags are more useful than lightweight tags because they store additional metadata such as the tagger name, creation date, and a descriptive message. This makes project version history more informative and easier to manage.

Creating a version tag after completing the merge workflow also reflects common collaborative development practice, where stable project versions are marked with release tags.

This workflow demonstrated the standard Git tagging process:

- check repository status
- create an annotated tag
- inspect the tag information
- push the tag to GitHub

Using clear version names such as `v1.0` also improves repository organisation and release tracking.

Result

The annotated tag `v1.0` was successfully created on the `main` branch and pushed to GitHub.

The terminal output confirmed that the new tag was synchronised with the remote repository. The tag now marks the merged and completed project version after resolving the merge conflict workflow.

8. Deleting testbranch Locally and Remotely

What I did

After merging `testbranch` into `main`, I checked the local branches using:

```
git branch
```

The output showed that I was currently on `main`, so it was safe to delete `testbranch`.

I deleted the local branch using:

```
git branch -d testbranch
```

I then checked the local branches again using:

```
git branch
```

The output confirmed that only `main` remained locally.

Finally, I deleted the remote branch from GitHub using:

```
git push origin --delete testbranch
```

Why I did this

The purpose of this step was to clean up the repository after the feature branch had been merged into `main`.

Once a branch has been merged and is no longer needed, deleting it keeps the repository simpler and easier to manage.

I used `git branch -d` instead of `git branch -D` because `testbranch` had already been merged. The `-d` option is safer because Git will prevent deletion if the branch has not been merged.

Deleting the remote branch is also necessary because deleting a local branch does not automatically remove the branch from GitHub.

Result

The branch `testbranch` was successfully deleted locally and remotely. The repository now only keeps `main` as the active branch.

The terminal output confirmed both the local deletion and the remote deletion of `testbranch`.

9. Showing the Commit Log in Condensed Form

What I did

To inspect the repository history, I displayed the Git commit log in condensed form using:

```
git log --oneline
```

The terminal output displayed a simplified version of the commit history, where each line contained:

- a shortened commit hash
- the corresponding commit message

The output included the commits created throughout the project workflow, including:

- the initial project commit
- branch development commits
- merge conflict workflow commits
- the final merge commit
- the annotated tag reference

Why I did this

The purpose of this step was to demonstrate how Git commit history can be inspected efficiently using a condensed log format.

Using `git log --oneline` makes the repository history easier to read because it provides a short summary of commits without displaying the full commit metadata such as author information and timestamps.

This command is useful for quickly reviewing project development history, checking workflow progress, and understanding the sequence of repository changes.

The condensed log also demonstrates the importance of using clear commit messages, because meaningful commit messages make the repository history easier to interpret.

Result

The repository commit history was successfully displayed in condensed form in the terminal.

The output clearly showed the complete Git workflow for the project, including branch development, merge conflict resolution, tagging, and final repository updates.

10. Undoing a Commit Without Losing Local Changes

What I did

On the `main` branch, I added a new section to `example.qmd` containing a simple plot using the built-in `cars` dataset.

After saving the file, I checked the repository status using:

```
git status
```

The output showed that `example.qmd` had been modified.

I then staged the modified file using:

```
git add example.qmd
```

After confirming that the file was staged, I created a commit using the following commit message:

```
Add simple plot section
```

Next, I demonstrated how to undo the most recent commit without losing the local changes by using:

```
git reset --soft HEAD~1
```

Finally, I checked the repository status again using:

```
git status
```

The output showed that `example.qmd` was still listed under “Changes to be committed”, which confirmed that the commit had been undone but the file changes were still preserved.

Why I did this

The purpose of this step was to demonstrate how Git can undo a commit while keeping the local work.

Using `git reset --soft HEAD~1` moves the branch pointer back by one commit but keeps the changes in the staging area. This is useful when a commit was made too early or needs to be adjusted, but the actual file changes should not be lost.

This workflow is safer than using a hard reset, because `git reset --hard` would remove both the commit and the local changes.

Result

The commit `Add simple plot section` was successfully undone. The local changes to `example.qmd` were not lost and remained staged.

This confirmed that the commit was removed from the current branch history while preserving the work in the local repository.